Informe ASR Aislamiento de carga por socio de distribución (Ivonne)

Un poco acerca de los cambios:

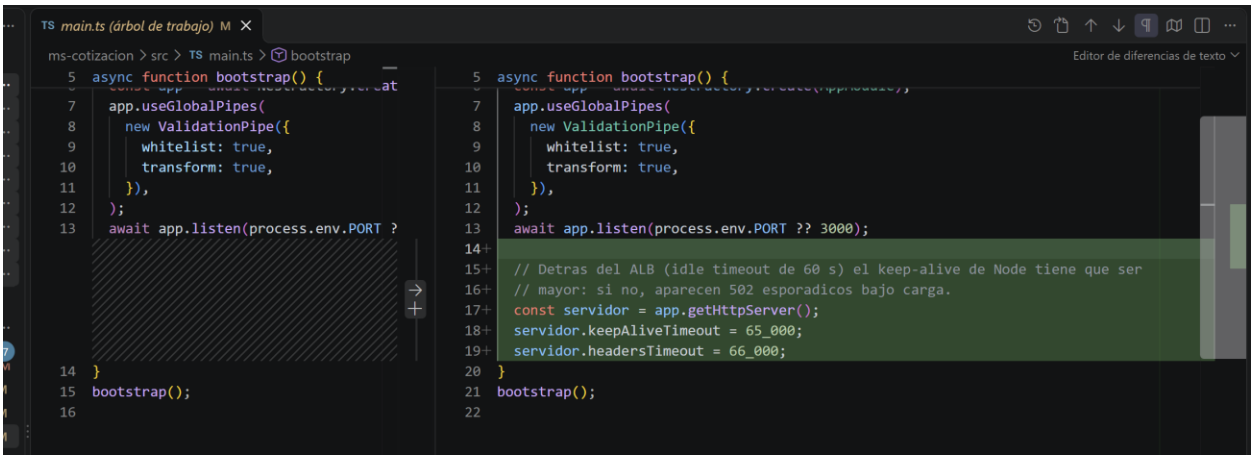
Antes tenía:

- Rate limiter (src/rate-limiter/). Era tu TODO.
    - Cada socio tiene su propio token bucket en Redis: 10 TPS con ráfaga de 20 por defecto. El exceso recibe 429 con Retry-After.
    - Es compartido entre réplicas. Si cada réplica tuviera su propio contador, con 2 a 4 réplicas el socio tendría 2 a 4 veces su cuota.
    - Un socio puede tener una cuota propia sin reiniciar nada: HSET cuota:<id> tps 50 rafaga 100.
- Caché (src/cache-clientes/). Guarda en Redis la verificación del cliente contra Identidad durante 60 s. Así, de cada cotización que repite cliente, solo la primera llega a Identidad.
- Variables para la corrida base: RATE_LIMIT_ENABLED=false y CACHE_ENABLED=false apagan las tácticas con la misma imagen.
- 50 socios de carga (Socio Carga 01…50, ids 00000000-0000-4000-8000-0000000000NN), además de Banco Aliado.
- Métricas: GET /metricas devuelve el porcentaje de aciertos de la caché y los 429 de cada socio. Además, cada réplica escribe esos contadores en el log cada 10 s.
- Keep-alive ajustado para que el ALB no devuelva 502 bajo carga.

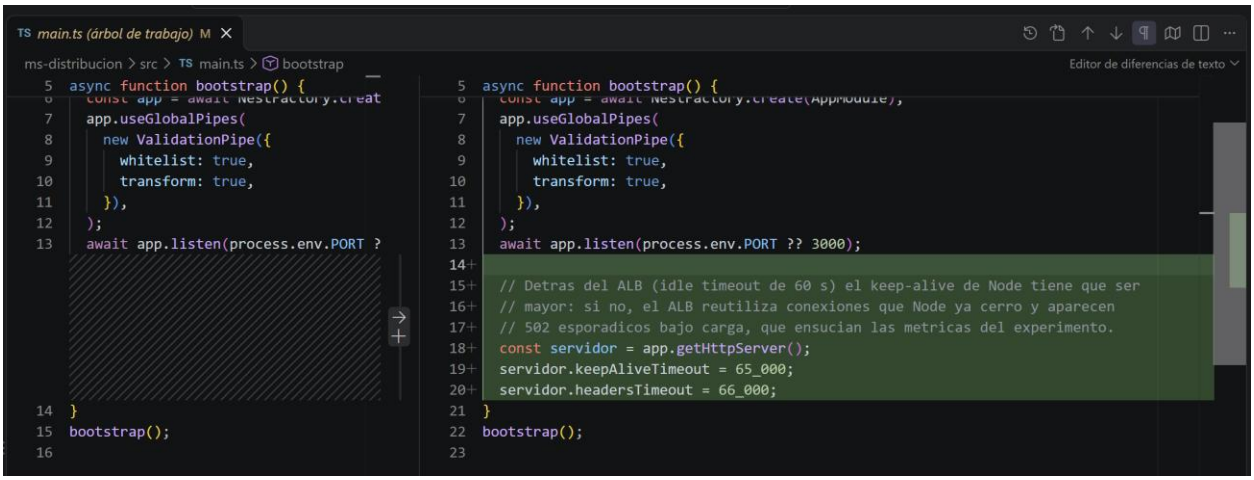
ms-cotizacion

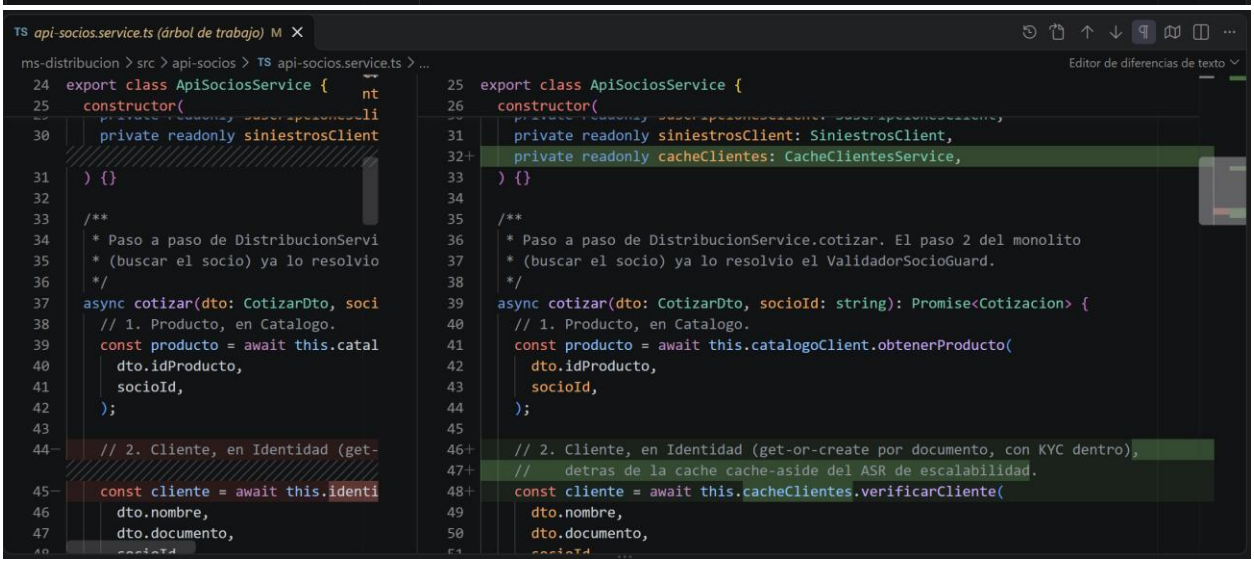
- Ahora acepta DB_SSL (RDS PostgreSQL 16 lo exige), DB_POOL_MAX (conexiones por réplica) y DB_SYNCHRONIZE, y tiene el mismo keep-alive.
- En local nada cambia, porque los valores por defecto son los de antes.

Cambios en cotización.



Cambios en distribución:



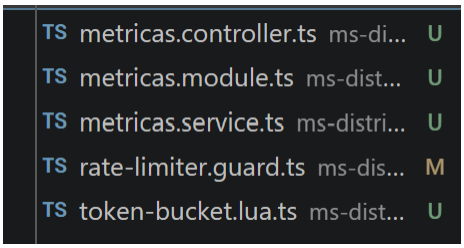


TS *cache-clientes.service.ts* U

ms-distribucion > src > cache-clientes > TS cache-clientes.service.ts > ...

```ts
import { createHash } from 'node:crypto';
import { Cliente, IdentidadClient } from '../clients/identidad.client';
import { MetricasService } from '../metricas/metricas.service';
import { RedisService } from '../redis/redis.service';

/**
 * ASR de escalabilidad (Ivonne). Tactica: cache cache-aside con TTL corto de
 * la verificacion del cliente contra Identidad (POST /clientes/verificacion),
 * la llamada a Identidad que hace CADA cotizacion.
 *
 *   1. busca en Redis    2. si no esta, llama a Identidad    3. guarda con TTL
 *
 * Es seguro cachearla: la verificacion es un get-or-create por documento, asi
 * que para el mismo documento Identidad devuelve siempre el mismo cliente.
 * Solo se cachean respuestas 200: un 400 de KYC o un 503 no se guardan.
 *
 * La clave es el SHA-256 del documento: el documento (dato personal) no queda
 * en claro en Redis.
 *
 * Si Redis no responde, se degrada a llamar directo a Identidad.
 * CACHE_ENABLED=false la apaga (corrida base del experimento, misma imagen).
 */
@Injectable()
```



TS token-bucket.lua.ts U

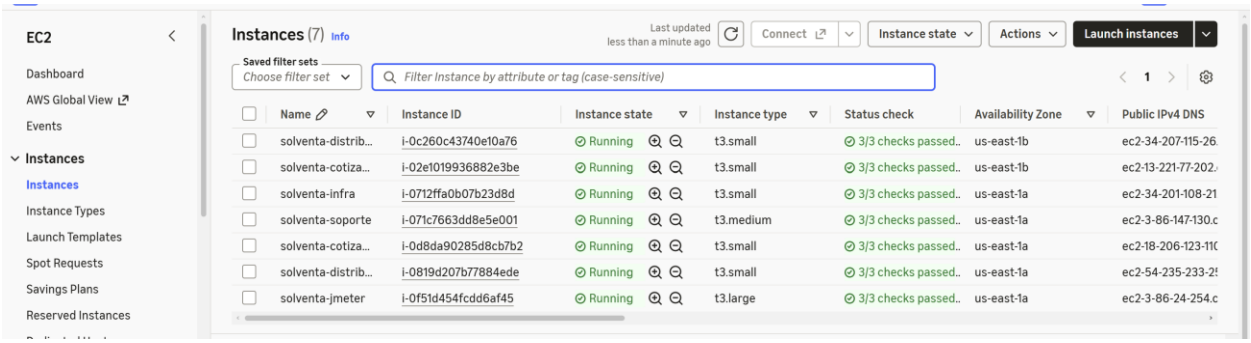
ms-distribucion > src > rate-limiter > TS token-bucket.lua.ts > ...

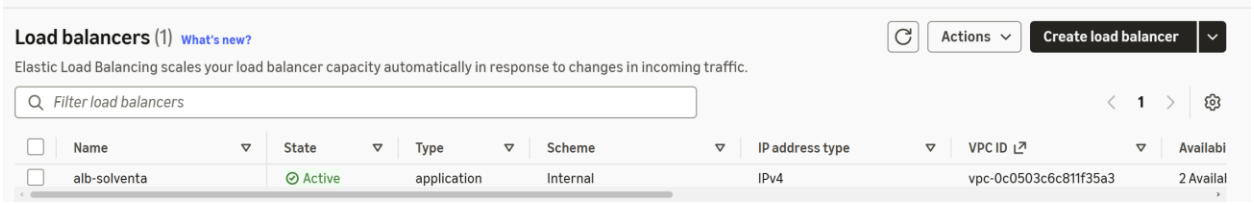
```
/**
 * Token bucket por socio, atomico dentro de Redis.
 *
 *   KEYS[1] = ratelimit:<socioId>  estado del bucket (tokens, ts)
 *   KEYS[2] = cuota:<socioId>      cuota propia opcional (hash tps, rafaga)
 *   ARGV[1] = tps por defecto      ARGV[2] = rafaga por defecto
 *   -> { permitido (1|0), tokensRestantes, tpsAplicado }
 *
 * Es en Redis y no en memoria pq hay varias replicas de Distribucion detras
 * del ALB (asg-distribucion 2..4). Un bucket por replica le daria al socio N
 * veces su cuota. Por que Lua: leer, recalcular y escribir tiene que ser una
 * sola operacion, si no dos replicas leen el mismo saldo y las dos lo gastan.
 * Usa el reloj de Redis (TIME) para que todas las replicas compartan la misma
 * referencia de tiempo aunque sus relojes difieran.
 */
export const TOKEN_BUCKET_LUA = `
local rate = tonumber(ARGV[1])
local burst = tonumber(ARGV[2])
local propia = redis.call('HMGET', KEYS[2], 'tps', 'rafaga')
if propia[1] then rate = tonumber(propia[1]) end
if propia[2] then burst = tonumber(propia[2]) end

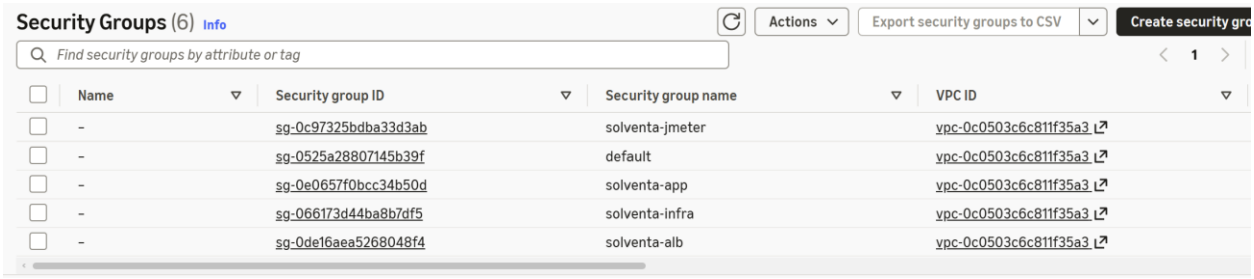
local t = redis.call('TIME')
```

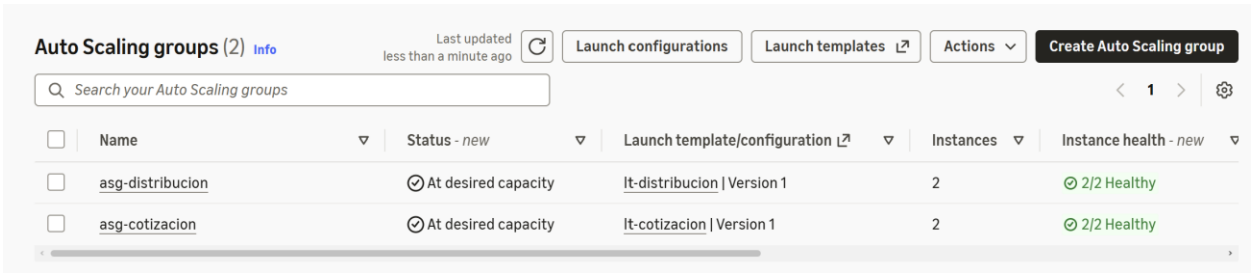
El despliegue en AWS se realizó desde CloudShell con un paquete que contiene la plantilla de CloudFormation (asr-escalabilidad.yaml), que describe toda la infraestructura del ASR; el script desplegar.sh, que crea el stack en la VPC por defecto; el script tacticas.sh, que cambia el stack entre la configuración sin tácticas (una instancia por grupo, sin rate limit ni caché) y con tácticas; el script metricas.sh, que exporta las métricas de CloudWatch de cada corrida; y la carpeta jmeter, con el plan de carga de 50 socios más el socio ruidoso, el script que lo ejecuta y el script que calcula el p95 y el p99 por fase y verifica el criterio.

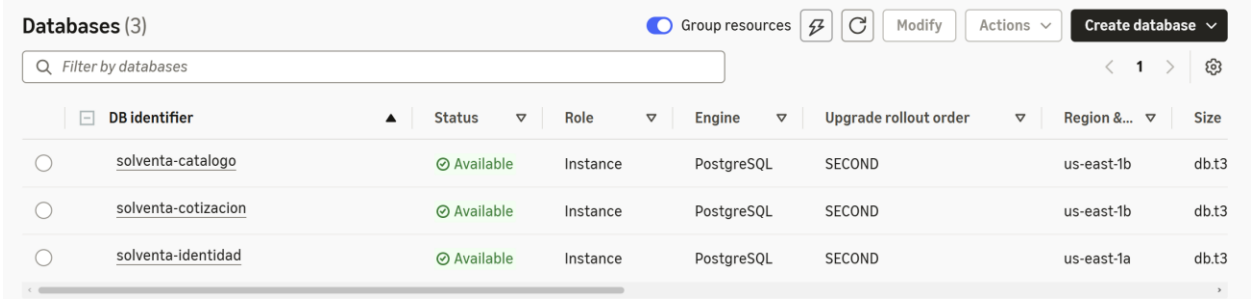
Y en aws se creó:

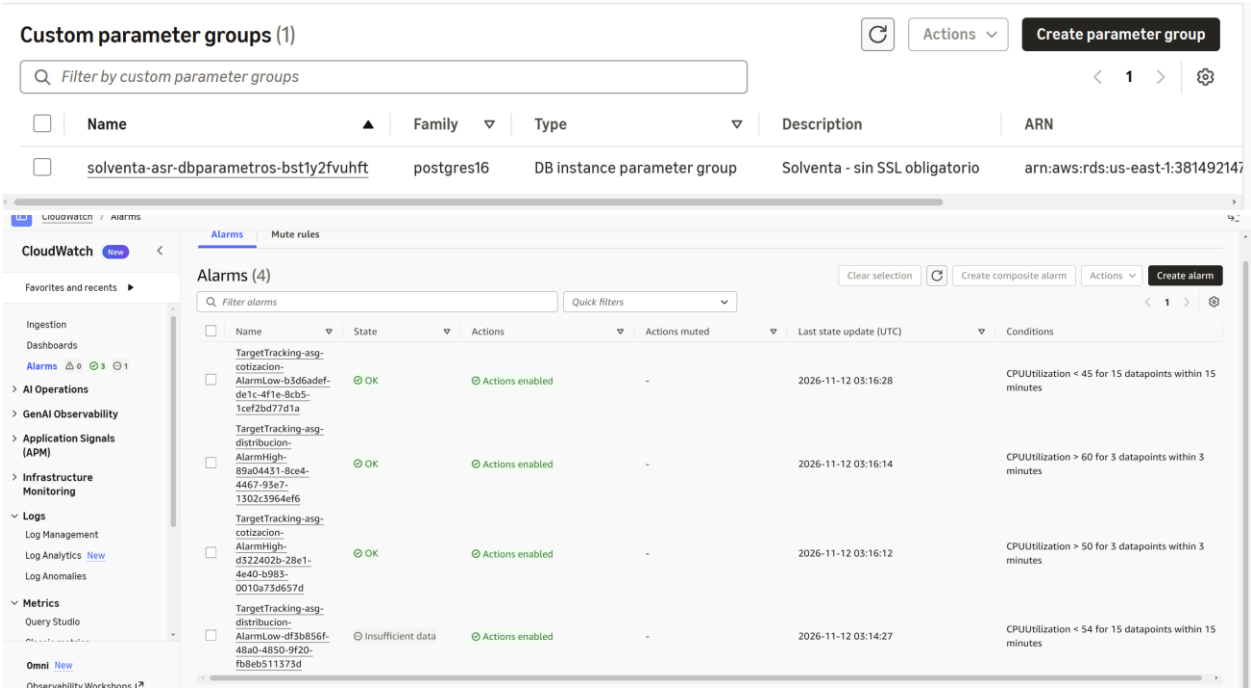



Load balancers (1) What's new?
Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.
Actions
Create load balancer
Filter load balancers
1
Name
State
Type
Scheme
IP address type
VPC ID
Availabi
alb-solventa
Active
application
Internal
IPv4
vpc-0c0503c6c811f35a3
2 Availal


Security Groups (6) Info
Actions
Export security groups to CSV
Create security gro
Find security groups by attribute or tag
1
Name
Security group ID
Security group name
VPC ID
- sg-0c97325bdba33d3ab solventa-jmeter vpc-0c0503c6c811f35a3
- sg-0525a28807145b39f default vpc-0c0503c6c811f35a3
- sg-0e0657f0bcc34b50d solventa-app vpc-0c0503c6c811f35a3
- sg-066173d44ba8b7df5 solventa-infra vpc-0c0503c6c811f35a3
- sg-0de16aea5268048f4 solventa-alb vpc-0c0503c6c811f35a3


Auto Scaling groups (2) Info
Last updated less than a minute ago
Launch configurations
Launch templates
Actions
Create Auto Scaling group
Search your Auto Scaling groups
1
Name
Status - new
Launch template/configuration
Instances
Instance health - new
asg-distribucion At desired capacity lt-distribucion | Version 1 2 2/2 Healthy
asg-cotizacion At desired capacity lt-cotizacion | Version 1 2 2/2 Healthy


Databases (3)
Group resources
Modify
Actions
Create database
Filter by databases
1
DB identifier
Status
Role
Engine
Upgrade rollout order
Region &...
Size
solventa-catalogo Available Instance PostgreSQL SECOND us-east-1b db.t3
solventa-cotizacion Available Instance PostgreSQL SECOND us-east-1b db.t3
solventa-identidad Available Instance PostgreSQL SECOND us-east-1a db.t3



Las tácticas implementadas y dónde encotnrarlas:

NOTA: En el video de Demo + Código se puede apreciar también

Rate limiting por socio (token bucket y 429): Está en ms-distribucion/src/rate-limiter/: rate-limiter.guard.ts decide si la petición pasa o recibe 429, y token-bucket.lua.ts contiene el bucket atómico en Redis.

Se aplica a todos los endpoints de los socios en api-socios.controller.ts, con @UseGuards(RateLimiterGuard, ValidadorSocioGuard), de modo que el exceso se rechaza antes de llegar a los demás servicios. Se evidencia en las respuestas 429 del socio ruidoso.

Caché cache-aside con TTL corto: Está en ms-distribucion/src/cache-clientes/cache-clientes.service.ts y se usa en el paso de verificación del cliente del método cotizar de api-socios.service.ts. Se evidencia en el porcentaje de aciertos que reporta GET /metricas.

La activación y desactivación de tácticas. Las variables RATE_LIMIT_ENABLED y CACHE_ENABLED, documentadas en .env.example, se pasan en AWS como parámetros del stack que modifica tacticas.sh.

Réplicas stateless detrás de la ALB: Distribución no guarda estado en memoria: los socios, las cuotas y la caché viven en Redis. En la infraestructura, los grupos asg-distribucion y asg-cotizacion están detrás del balanceador alb-solventa, y sus target groups muestran las instancias saludables.

Autoescalado horizontal independiente. En asr-escalabilidad.yaml, AsgDistribucion con PoliticaDistribucion y AsgCotizacion con PoliticaCotizacion definen un grupo y una política de CPU por servicio.

Esto se evidencia en la actividad de cada grupo y en el número de instancias en servicio a lo largo del tiempo (en el video Demo + Código se ve la demostración).

Base de datos por servicio: La infraestructura crea tres instancias RDS separadas (cotización, catálogo e identidad), y cada servicio recibe su propio DB_HOST.

Dimensionamiento del pool de conexiones: En ms-cotizacion/src/app.module.ts, el pool se limita con DB_POOL_MAX; en AWS, 6 réplicas por 10 conexiones suman 60, por debajo del límite de la base db.t3.micro. Se evidencia en la métrica DatabaseConnections de RDS.

Para el experimento:

Paso a la configuración sin tácticas

```
~ $    cd ~/solventa-asr-aws
solventa-asr-aws $    ./tacticas.sh sin

Waiting for changeset to be created..
Waiting for stack create/update to complete
Successfully created/updated stack - solventa-asr
Listo. Estado de los targets:
  tg-distribucion: unhealthy
  tg-cotizacion: healthy
  tg-catalogo: healthy
solventa-asr-aws $
solventa-asr-aws $ for tg in tg-distribucion tg-cotizacion tg-catalogo; do
>   arn=$(aws elbv2 describe-target-groups --names $tg --query 'TargetGroups[0].TargetGroupArn' --output text)
>   echo "$tg: $(aws elbv2 describe-target-health --target-group-arn $arn --query 'TargetHealthDescriptions[].TargetHealth.State' --output text)"
> done
tg-distribucion: healthy
tg-cotizacion: healthy
tg-catalogo: healthy
```

Corrida sin tácticas

```
Tidying up ...    @ 2026 Nov 12 05:14:19 UTC (1794460459504)
... end of run
Fin: 2026-11-12T05:14:20Z
Archivo: resultados/sin-tacticas.jtl   peticiones: 370560   duracion: 1320 s

fase                    TPS  p95 normales  p99 normales  errores normales  429 ruidoso  201 ruidoso
linea base               50         82 ms        369 ms                 0            0         1198
ramp-up                 256        837 ms       1231 ms                 0            0         2870
estable                 354       1119 ms       1958 ms               150            0         7990
  (socio ruidoso)       357       1226 ms       1943 ms                15            0         4594
bajada                  251        861 ms       1401 ms                 0            0         2824

Codigos de respuesta:
  socio 01 (ruidoso)   201: 14882  500: 4
  socios 02-50         201: 355524  500: 146  503: 4

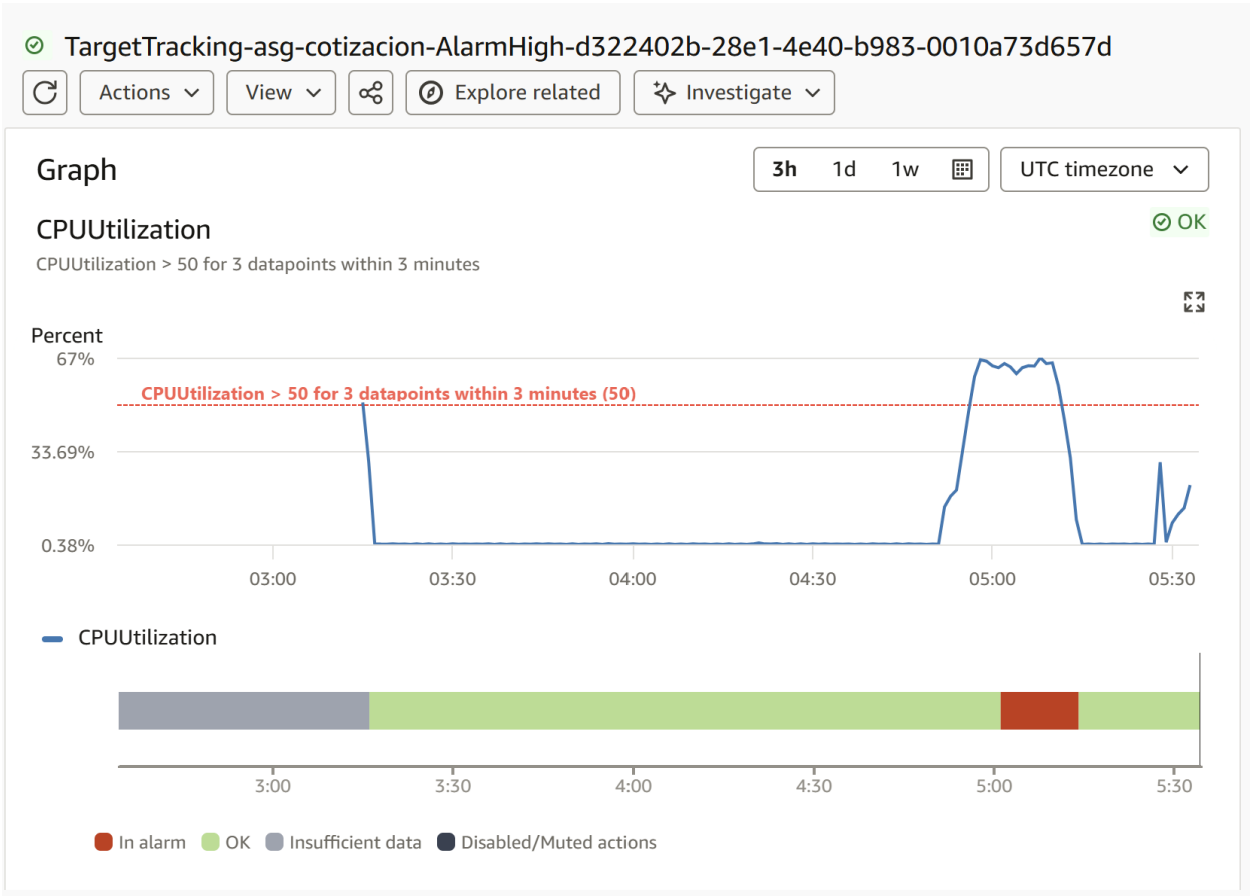
Criterio del ASR (socios 02-50, ramp-up y estable: p95 <= 250 ms, p99 <= 500 ms, 0 errores): NO CUMPLE
```

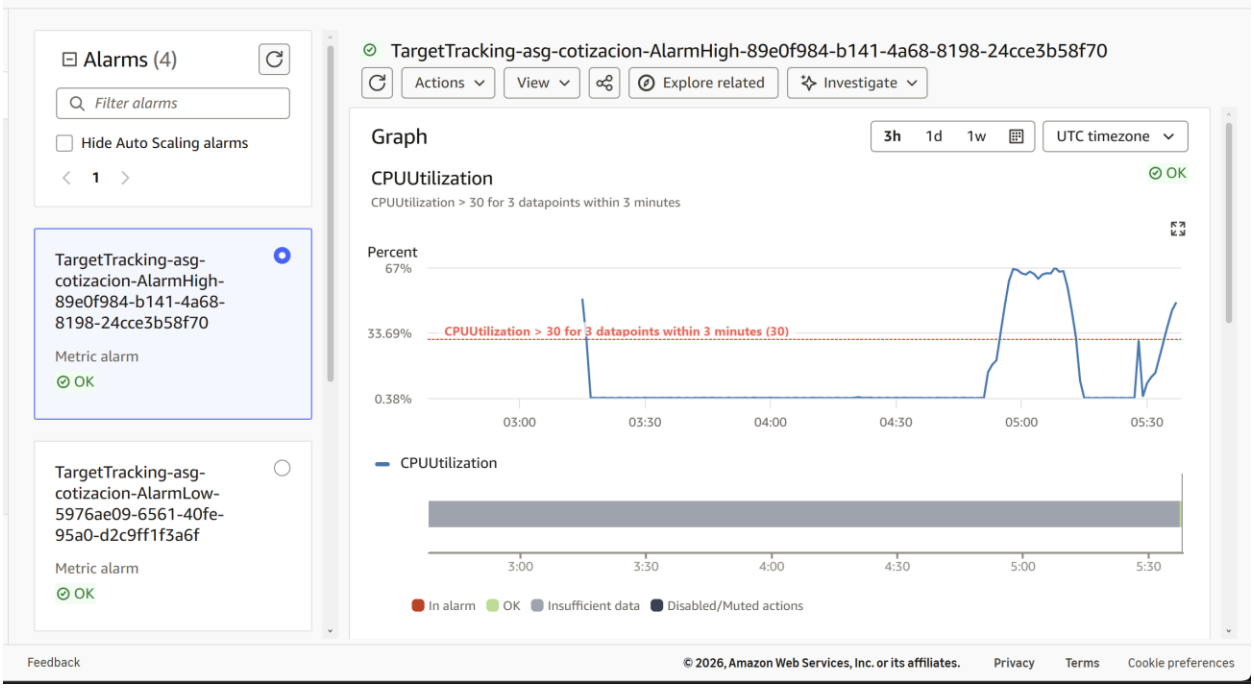
Paso a la configuración con tácticas:

```
solventa-asr-aws $ cd ~/solventa-asr-aws
solventa-asr-aws $ ./tacticas.sh con

Waiting for changeset to be created..
Waiting for stack create/update to complete
Successfully created/updated stack - solventa-asr
Listo. Estado de los targets:
  tg-distribucion: draining     unhealthy       initial
  tg-cotizacion: unhealthy      healthy
  tg-catalogo: unhealthy        healthy
solventa-asr-aws $ for tg in tg-distribucion tg-cotizacion tg-catalogo; do
>   arn=$(aws elbv2 describe-target-groups --names $tg --query 'TargetGroups[0].TargetGroupArn' --output text)
>   echo "$tg: $(aws elbv2 describe-target-health --target-group-arn $arn --query 'TargetHealthDescriptions[].TargetHealth.State' --output text)"
> done
tg-distribucion: healthy        healthy
tg-cotizacion: healthy  healthy
tg-catalogo: healthy    healthy
```

Corrida con tácticas:






```
... end of run
Fin: 2026-11-12T05:51:49Z
Archivo: resultados/con-tacticas.jtl   peticiones: 478693   duracion: 1320 s

fase                  TPS  p95 normales  p99 normales  errores normales  429 ruidoso  201 ruidoso
linea base             50         64 ms        386 ms                 0            0         1199
ramp-up               274         79 ms        605 ms                 0            0         2994
estable               514        166 ms        881 ms                24        10778         5977
  (socio ruidoso)     584        169 ms        787 ms                21        10748         1217
bajada                274        154 ms        769 ms                 0            0         2990

Codigos de respuesta:
  socio 01 (ruidoso)   201: 13160  429: 10778  500: 1
  socios 02-50         201: 454730  500: 21  503: 3

Criterio del ASR (socios 02-50, ramp-up y estable: p95 <= 250 ms, p99 <= 500 ms, 0 errores): NO CUMPLE
```

RESULTADOS DEL EXPERIMENTO:

ANÁLISIS DE RESULTADOS:

Verificación de la hipótesis:

La hipótesis se confirma parcialmente. Las tácticas resuelven la saturación del sistema y aíslan a los socios entre sí, el p95 cumple el criterio en todas las fases, el sistema sostiene los ~500 TPS y el exceso del socio ruidoso no afecta a los demás. Sin embargo, el p99 y una pequeña tasa de errores no cumplen el criterio.

¿Por qué el sistema se saturó sin tácticas?:

Cada microservicio corre en un único proceso de Node, que ejecuta el código de la aplicación en un solo hilo y por lo tanto aprovecha un solo núcleo.

Con una sola réplica de Cotización, al crecer la carga ese núcleo se saturó (65 % de CPU en una instancia de 2) y las peticiones empezaron a hacer fila, debían esperar a las anteriores, lo que multiplicó el p95. Como JMeter envía la siguiente petición de cada hilo solo cuando recibe la respuesta anterior, la lentitud también redujo la carga efectivamente enviada (354 de ~500 TPS), lo cual es típico de saturación. Sin rate limit, el socio ruidoso agregó ~90 TPS a un sistema ya saturado, y en su ventana aumentaron tanto la latencia como los errores de los demás socios. Es justamente el problema que el ASR busca evitar.

Efectos de las tácticas:

- Réplicas y autoescalado:
  Con 2 réplicas desde el inicio, la carga se repartió y ninguna instancia se saturó durante el ramp-up, el p95 bajó de 837 a 79 ms y el sistema alcanzó 514 TPS.
  El autoescalado agregó una tercera réplica de Cotización cuando su CPU superó el objetivo, mientras Distribución se mantuvo en dos, lo que confirma que la capacidad se agrega SOLO donde está la carga, lo cual se alínea con nuestro objetivo.

- Rate limit por socio:
  El exceso del socio ruidoso se rechazó en el borde con 429, antes de llegar a Catálogo, Identidad, Cotización o las bases de datos. Por eso la latencia de los demás socios no cambió durante su ventana. Como el contador de cada socio vive en Redis y se actualiza con un script atómico, la cuota se respetó aunque el ALB repartiera las peticiones entre varias réplicas.

- Caché de verificación de clientes:
  Hubo un hit ratio de ~96 %, la gran mayoría de las cotizaciones no necesitó llamar a Identidad, lo que reduce la carga sobre un componente que no escala.

- Pool de conexiones por réplica:
  Limitar a 10 las conexiones de cada réplica de Cotización garantiza que, incluso con 6 réplicas, la base no supere su capacidad de conexiones.

¿Por qué falló el p99?

1. El autoescalado reacciona más lento que el ramp-up, porque la alarma necesita 3 minutos sobre el objetivo y una instancia nueva tarda 2 a 3 minutos en quedar lista, mientras el ramp-up dura 5 minutos. Durante esa transición las réplicas existentes trabajaron cerca de su límite.

Además, el objetivo inicial de 50 % estaba mal calibrado para un proceso de un solo hilo y se corrigió a 30 % con la corrida en curso (esto también es mencionado y mostrado en el video de Demo + Código).

2. Hay componentes compartidos que no escalan, como Identidad (1 sola instancia), Redis (1 sola instancia) y bases de datos db.t3.micro participan en cada cotización, y cualquier demora en ellos cae en el 1 % más lento.

¿Por qué hubo errores?

21 de los 24 errores ocurrieron durante la ventana del socio ruidoso.

Rechazar con 429 es barato, pero no es del todo gratis, Distribución igual recibe y procesa cada petición del ruidoso para decidir el rechazo. En esa ventana entraron ~584 TPS a Distribución, cuyo objetivo de CPU seguía en 60 % y por eso no escaló. Con su hilo muy ocupado, algunas consultas a Redis del validador de socios superaron su tiempo límite y respondieron 500.

Los 3 errores 503 corresponden a llamadas a otros servicios que no respondieron a tiempo. A pesar de ello, los errores se redujeron en 84 % frente a la corrida sin tácticas.

Recomendaciones:

- Para cumplir el p99 y los 0 errores se propone: calibrar desde el inicio los objetivos de CPU para un proceso de un solo hilo o escalar por peticiones por instancia.
- Ejecutar dos procesos de Node por instancia para aprovechar los dos núcleos
- Pre-escalar antes del horario pico, dado que el ramp-up es más rápido que la reacción del autoescalads, es decir, hacerlo predictivo
- Escalar los componentes compartidos (réplicas de Identidad, Redis administrado y bases de mayor capacidad)

CONCLUSIONES:

El experimento muestra que las tácticas elegidas atacan correctamente el problema del ASR. Sin ellas, la carga de 50 socios satura el sistema y el exceso de un socio ruidoso degrada a todos. Con ellas, el p95 de los socios que respetan su cuota bajó de 1119 a 166 ms, el sistema sostuvo los ~500 TPS y el socio ruidoso quedó completamente aislado: 10 778 rechazos en el borde y ningún 429

para los demás. Además, cada servicio escaló de forma independiente según su propia carga. Por lo tanto, la hipótesis se confirma en su parte central, aunque el criterio no se cumple por completo: el p99 (881 ms) y 24 errores (0,005 %) siguen sin cumplir objetivo.

Más allá del resultado, quedan varios aprendizajes:

- Aislar a los socios con una cuota individual en el borde es más efectivo que solo agregar capacidad, y esa cuota solo funciona entre réplicas si su estado es compartido y atómico.
- El autoescalado no es instantáneo y su umbral debe calibrarse según la tecnología: en Node, un 50 % de CPU en dos núcleos ya es saturación.
- La cola depende de los componentes que no escalan, de las transiciones del autoescalado y de los arranques en frío. Y rechazar es barato pero no gratis, por lo que el borde también debe dimensionarse y escalar.

En consecuencia, se deberían adoptar las tácticas con modificaciones: umbrales de autoescalado calibrados para Node, dos procesos por instancia, pre-escalado antes del pico (predictivo), escalado de los componentes compartidos y manejo del tiempo límite de Redis en el validador. El siguiente paso es repetir el experimento con estos cambios para verificar el p99 $\leq$ 500 ms y los 0 errores.